

可 scan 长依赖链段占比

这是你的创新点能不能继续做下去的关键。

把第一个创新点内容和第一步统计数据集的要求编写成文档给我，我拿去生成代码（目前最小版本就行）

我会把它写成一份“给 Codex/代码生成器看的需求文档”，只保留最小可用版本：先统计结构，不写 GPU kernel，不做数值求解。

下面是一份可以直接给 **Codex / 代码生成器** 用的需求文档。

这是**最小可用版本**，目标只做一件事：

统计数据集中有多少 SpTRSV 依赖结构适合做“长依赖链 → 并行前缀扫描”变换。

文档：SpTRSV 可扫描长依赖链统计工具需求

1. 背景与目标

本项目研究 SpTRSV，也就是稀疏三角矩阵求解：

$$Lx = b$$

其中 L 是下三角矩阵。

SpTRSV 的主要瓶颈之一是行与行之间存在依赖关系。如果：

$$L[i, j] \neq 0 \text{ 且 } j < i$$

那么第 i 行的未知量 x_i 依赖第 j 行的未知量 x_j 。

也就是说：

$$x_j \rightarrow x_i$$

如果依赖路径很长，传统 level-based SpTRSV 需要很多层同步；sync-free 方法虽然减少同步，但线程可能长时间等待依赖完成。

本创新点尝试把一部分长依赖链改写成并行前缀扫描。

2. 第一个创新点：Dependency Chain → Parallel Scan

中文名称：

把 SpTRSV 中的长依赖链转化为并行前缀扫描

更严谨的中文名称：

面向 SpTRSV 长依赖路径的仿射扫描变换

英文名称可以暂定为：

Affine-Scan Transformation for Long Dependency Paths in SpTRSV

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

其中：

- **Affine**：仿射，指 $\alpha x + \beta$ 这种形式。
- **Scan**：前缀扫描，也叫 prefix scan。
- **Long Dependency Path**：长依赖路径。

3. 数学形式

对于一条近似链状依赖：

$x_p \rightarrow x_i$

如果第 i 行主要依赖链内前驱 x_p ，并且其他依赖项都已经求解完成，则原方程：

$$L[i,i] * x_i + L[i,p] * x_p + \sum L[i,j] * x_j = b_i$$

可以写成：

$$x_i = \alpha_i * x_p + \beta_i$$

其中：

$$\alpha_i = -L[i,p] / L[i,i]$$

$$\beta_i = (b_i - \sum L[i,j] * x_j) / L[i,i]$$

这里的 j 表示链外已经求解完成的依赖项。

因此，每一行可以看作一个仿射函数：

$$f_i(x) = \alpha_i * x + \beta_i$$

链上求解：

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots \rightarrow x_n$

可以变成函数复合：

$f_1, f_2, f_3, \dots, f_n$

两个仿射函数复合仍然是仿射函数：

$$f_i(x) = \alpha_i x + \beta_i$$

$$f_j(x) = \alpha_j x + \beta_j$$

$$f_j(f_i(x)) = \alpha_j \alpha_i x + \alpha_j \beta_i + \beta_j$$

所以可以把每个函数表示成一个二元组：

(α_i, β_i)

复合规则是：

$$(\alpha_j, \beta_j) \circ (\alpha_i, \beta_i)$$

$$= (\alpha_j * \alpha_i, \alpha_j * \beta_i + \beta_j)$$

该复合操作满足结合律，因此可以使用并行前缀扫描。

4. 本阶段只做什么

本阶段只做数据集结构统计。

不做：

1. 不写 GPU kernel
2. 不真正求解 $Lx=b$
3. 不比较性能
4. 不做数值误差分析
5. 不改 cuSPARSE 或其他 SpTRSV 求解器

只做：

1. 读取 .mtx 稀疏矩阵
2. 提取下三角依赖图
3. 计算 level 深度
4. 统计长依赖路径是否存在
5. 统计有多少行可以近似转化为 scan
6. 输出 CSV 结果

5. 输入数据要求

输入是一个目录，里面包含多个 .mtx 矩阵文件。

示例目录：

```
datasets/
  matrix1.mtx
  matrix2.mtx
  matrix3.mtx
```

每个矩阵要求：

1. Matrix Market 格式，也就是 .mtx 文件
2. 矩阵必须是方阵
3. 默认分析下三角部分
4. 对角线用于计算，但依赖图中不把对角线当依赖边

本阶段默认只分析：

lower triangular matrix

也就是下三角矩阵。

6. 依赖图定义

对于下三角矩阵 L ：

如果 $L[i,j] \neq 0$ 且 $j < i$
那么存在依赖边 $j \rightarrow i$

其中：

j 是前驱行
 i 是当前行

每一行的前驱集合定义为：

$\text{pred}[i] = \{ j \mid L[i,j] \neq 0 \text{ 且 } j < i \}$

每一行的依赖数量定义为：

```
in_degree[i] = len(pred[i])
```

7. level 定义

`level[i]` 表示第 `i` 行所在的依赖层级。

定义：

如果 `pred[i]` 为空：
`level[i] = 1`

否则：
`level[i] = 1 + max(level[j])`，其中 `j ∈ pred[i]`

也就是说，一行的 `level` 等于它所有前驱中最大 `level` 加 1。

`max_level` 表示整个矩阵的最长依赖路径长度：

```
max_level = max(level[i])
```

8. 普通长依赖路径统计指标

这部分回答：

数据集中是否存在长依赖路径？

每个矩阵需要统计：

```
max_level
critical_depth_ratio
deep_row_ratio_5pct
deep_row_ratio_10pct
```

定义如下。

8.1 max_level

```
max_level = max(level[i])
```

表示最长依赖路径长度。

8.2 critical_depth_ratio

```
critical_depth_ratio = max_level / n_rows
```

表示最长依赖路径占矩阵行数的比例。

8.3 deep_row_ratio_5pct

```
deep_row_ratio_5pct =
count(level[i] >= 0.05 * n_rows) / n_rows
```

表示 `level` 深度超过矩阵规模 5% 的行比例。

8.4 deep_row_ratio_10pct

```
deep_row_ratio_10pct =
count(level[i] >= 0.10 * n_rows) / n_rows
```

表示 level 深度超过矩阵规模 10% 的行比例。

9. 可 scan 行的最小判定规则

本阶段采用一个保守的静态近似规则。

对于某一行 i ，如果它有前驱：

```
pred[i] != empty
```

找到它所有前驱里 level 最大的前驱。

设：

```
max_pred_level = max(level[j]), j ∈ pred[i]
```

如果只有一个前驱 p 满足：

```
level[p] == max_pred_level
```

那么认为 p 是第 i 行的唯一关键前驱。

此时可以把第 i 行近似写成：

$$x_i = \alpha_i * x_p + \beta_i$$

原因是：

1. p 是唯一最深的未解析前驱
2. 其他前驱的 level 更低，可以近似认为已经提前求解完成
3. 因此其他前驱可以吸收到 β_i 里面

于是：

i 是 scan-eligible row

中文：

i 是可扫描行

如果有多个前驱同时处在最大 level：

```
level[p1] == level[p2] == max_pred_level
```

那么第 i 行同时依赖多个未解析关键前驱，不能直接写成一元仿射递推。

这种行记为：

```
invalid_multi_unresolved row
```

中文：

多关键未解析前驱行

10. 可 scan 边定义

如果第 i 行是可扫描行，并且它的唯一关键前驱是 p ，则建立一条可扫描边：

$p \rightarrow i$

也就是：

`scan_parent[i] = p`

如果第 i 行不可扫描，则：

`scan_parent[i] = -1`

通过所有 `scan_parent` 可以得到一个由可扫描边组成的图。

11. 可 scan 链段提取规则

根据 `scan_parent` 构建 `scan_children`：

`scan_children[p].append(i)`

然后从这个图中提取链段。

最小版本可以使用如下规则：

1. 每条 `scan` 边 $p \rightarrow i$ 表示 i 可以接在 p 后面做仿射递推。
2. 如果一个节点只有一个 `scan child`，则可以继续延长链。
3. 如果一个节点有多个 `scan child`，说明发生分叉，需要从每个 `child` 重新开始一条链。
4. 如果一个节点没有 `scan child`，链结束。

链段长度定义为：

该链段中 `scan-eligible rows` 的数量

注意：

链段的起点前驱 p 可以作为边界值，不一定计入 `scan-eligible rows`。

例如：

`x0 -> x3 -> x7 -> x10`

如果可扫描行是：

`x3, x7, x10`

那么链段长度为：

3

12. 可 scan 统计指标

这部分回答：

这些长依赖结构里，有多少可以转化成 `parallel scan`？

每个矩阵需要统计：

`scan_chain_count`
`scan_eligible_row_ratio`
`max_scan_chain_length`
`avg_scan_chain_length`

```
scan_chain_len_ge_32
scan_chain_len_ge_64
invalid_multi_unresolved_ratio
```

具体定义如下。

12.1 scan_chain_count

`scan_chain_count` = 提取出的可 scan 链段数量

12.2 scan_eligible_rows

`scan_eligible_rows` = `count(scan_parent[i] != -1)`

12.3 scan_eligible_row_ratio

`scan_eligible_row_ratio` = `scan_eligible_rows / n_rows`

表示矩阵中有多少比例的行可以被仿射 scan 处理。

12.4 max_scan_chain_length

`max_scan_chain_length` = `max(length of scan chains)`

表示最长可扫描链段长度。

12.5 avg_scan_chain_length

`avg_scan_chain_length` = `average(length of scan chains)`

表示平均可扫描链段长度。

如果没有可扫描链段，则：

```
avg_scan_chain_length = 0
max_scan_chain_length = 0
```

12.6 scan_chain_len_ge_32

```
scan_chain_len_ge_32 =
count(scan_chain_length >= 32)
```

表示长度大于等于 32 的可扫描链段数量。

12.7 scan_chain_len_ge_64

```
scan_chain_len_ge_64 =
count(scan_chain_length >= 64)
```

表示长度大于等于 64 的可扫描链段数量。

12.8 invalid_multi_unresolved_ratio

统计因为多个关键前驱而不能 scan 的比例。

定义：

```
invalid_multi_unresolved_rows =
count( pred[i] 非空, 并且最大 level 前驱数量 > 1 )

invalid_multi_unresolved_ratio =
invalid_multi_unresolved_rows / n_rows
```

这个指标用来说明：

有多少行不满足一元仿射递推条件。

13. 最小版本 CSV 输出字段

每个矩阵输出一行。

CSV 字段如下：

```
matrix_name
n_rows
nnz

max_level
critical_depth_ratio
deep_row_ratio_5pct
deep_row_ratio_10pct

scan_chain_count
scan_eligible_rows
scan_eligible_row_ratio
max_scan_chain_length
avg_scan_chain_length
scan_chain_len_ge_32
scan_chain_len_ge_64
invalid_multi_unresolved_rows
invalid_multi_unresolved_ratio
```

字段解释：

字段	含义
matrix_name	矩阵文件名
n_rows	矩阵行数
nnz	矩阵非零元数量
max_level	最长依赖路径长度
critical_depth_ratio	$\text{max_level} / \text{n_rows}$
deep_row_ratio_5pct	深度超过 5% * n_rows 的行比例
deep_row_ratio_10pct	深度超过 10% * n_rows 的行比例
scan_chain_count	可扫描链段数量
scan_eligible_rows	可扫描行数量
scan_eligible_row_ratio	可扫描行数量 / 总行数
max_scan_chain_length	最长可扫描链段长度
avg_scan_chain_length	平均可扫描链段长度
scan_chain_len_ge_32	长度 ≥ 32 的可扫描链段数量
scan_chain_len_ge_64	长度 ≥ 64 的可扫描链段数量
invalid_multi_unresolved_rows	多关键前驱导致不能 scan 的行数
invalid_multi_unresolved_ratio	多关键前驱行数量 / 总行数

14. 程序输入输出要求

14.1 输入参数

希望脚本支持命令行参数：

Bash

```
python analyze_scan_chains.py --dir ./datasets --out scan_chain_stats.csv
```

参数：

--dir

输入 .mtx 文件所在目录

--out

输出 CSV 文件路径

可选参数：

--triangle lower

本阶段默认：

lower

14.2 输出文件

输出一个 CSV 文件：

scan_chain_stats.csv

每个矩阵一行。

15. 程序整体流程

伪代码如下：

for each .mtx file in input directory:

1. 读取矩阵 A
2. 检查是否为方阵
3. 提取下三角依赖，不包含对角线
4. 构建 pred[i]
5. 计算 level[i]
6. 计算普通长依赖路径指标
7. 对每一行判断是否 scan-eligible
8. 构建 scan_parent 和 scan_children
9. 提取 scan chains
10. 统计 scan chain 指标
11. 写入 CSV

16. 关键函数设计建议

建议代码中至少包含这些函数。

16.1 load_matrix

Python

```
def load_matrix(path):
    """
    读取 .mtx 文件, 返回 scipy sparse CSR 矩阵。
    """
```

16.2 build_predecessors

Python

```
def build_predecessors(A):
    """
    输入 CSR 矩阵 A。
    返回 pred 列表。
    pred[i] = 所有满足 j < i 且 A[i,j] != 0 的列号 j。
    """
```

16.3 compute_levels

Python

```
def compute_levels(pred):
    """
    根据 pred 计算每一行的 level。
    下三角矩阵按 i 从 0 到 n-1 扫描即可。
    """
```

16.4 find_scan_parents

Python

```
def find_scan_parents(pred, levels):
    """
    对每一行 i:
        如果 pred[i] 为空:
            scan_parent[i] = -1
        否则:
            找到 pred[i] 中 level 最大的前驱集合 max_preds
            如果 len(max_preds) == 1:
                scan_parent[i] = max_preds[0]
            否则:
                scan_parent[i] = -1
                invalid_multi_unresolved_rows += 1

    返回:
        scan_parent
        invalid_multi_unresolved_rows
    """
```

16.5 extract_scan_chains

Python

```
def extract_scan_chains(scan_parent):
    """
    根据 scan_parent 构建 scan_children。
    提取可扫描链段长度列表 chain_lengths。
    """
```

```

返回：
    chain_lengths
    """

```

16.6 analyze_matrix

Python

```

def analyze_matrix(path):
    """
    分析单个矩阵，返回一个 dict。
    dict 中包含 CSV 所需的所有字段。
    """

```

17. 最小版本依赖

Python 依赖：

Bash

```

pip install numpy scipy

```

如果 Ubuntu 系统不能直接 pip install，可以使用虚拟环境：

Bash

```

python3 -m venv venv
source venv/bin/activate
pip install numpy scipy

```

18. 代码正确性检查用的小例子

可以在代码中额外提供一个小测试矩阵。

例如下三角结构：

```

row 0: diagonal only
row 1: depends on 0
row 2: depends on 1
row 3: depends on 2

```

依赖图：

```

0 -> 1 -> 2 -> 3

```

期望：

```

level = [1, 2, 3, 4]
max_level = 4
scan_eligible_rows = 3
max_scan_chain_length = 3

```

另一个例子：

```

row 0: diagonal only
row 1: diagonal only
row 2: depends on 0 and 1

```

其中 row 2 同时依赖两个相同 level 的前驱：

```

0 -> 2
1 -> 2

```

期望：

```
level[0] = 1
level[1] = 1
level[2] = 2
```

row 2 有两个最大 level 前驱

所以 row 2 不可 scan

```
invalid_multi_unresolved_rows = 1
```

19. 最终希望看到的输出示例

CSV 示例：

```
matrix_name,n_rows,nnz,max_level,critical_depth_ratio,deep_row_ratio_5pct,deep_row_ratio_10pct,scan_
chain_count,scan_eligible_rows,scan_eligible_row_ratio,max_scan_chain_length,avg_scan_chain_length,s
can_chain_len_ge_32,scan_chain_len_ge_64,invalid_multi_unresolved_rows,invalid_multi_unresolved_rati
o
A.mtx,100000,700000,12000,0.12,0.31,0.18,250,42000,0.42,512,48.6,80,21,3000,0.03
B.mtx,50000,300000,300,0.006,0.00,0.00,120,8000,0.16,24,5.3,0,0,2000,0.04
```

20. 这一步实验完成后要回答的问题

这个统计脚本跑完后，我们要能回答：

1. 数据集中是否存在深层依赖路径？
2. 哪些矩阵的 `critical_depth_ratio` 比较大？
3. 哪些矩阵有较高的 `deep_row_ratio`？
4. 数据集中有多少行满足 `scan-eligible` 条件？
5. 可 `scan` 链段是否足够长？
6. 长度 ≥ 32 或 ≥ 64 的可 `scan` 链段有多少？
7. 有多少行因为多个关键前驱而不能 `scan`？

21. 本阶段不要求完美

本阶段的统计是最小版本，判定规则是保守近似：

唯一最大 level 前驱 = 唯一未解析关键前驱

这个规则不一定覆盖所有可 `scan` 情况，但它有两个优点：

1. 实现简单
2. 统计结果偏保守，不容易夸大创新点适用范围

后续可以进一步扩展：

1. 更精确地区分链外依赖是否已经 `ready`
2. 统计 α 的数值分布
3. 支持上三角矩阵
4. 支持链段收益估计
5. 支持真正生成 `scan kernel`

22. 给 Codex 的一句话任务

可以把下面这段直接发给 Codex：

请根据这份需求文档实现一个 Python 脚本 `analyze_scan_chains.py`。脚本读取指定目录下的 `.mtx` 稀疏矩阵文件，默认分析下三角依赖结构，计算每个矩阵的 `level` 深度、长依赖路径指标，以及基于“唯一最大 `level` 前驱”的可扫描链段指标。最后把每个矩阵的统计结果输出到 CSV。要求代码结构清晰，包含 `load_matrix`、`build_predecessors`、`compute_levels`、`find_scan_parents`、`extract_scan_chains`、`analyze_matrix` 等函数，并能通过命令行参数 `--dir` 和 `--out` 运行。